Proyecto Final de Probabilidad: Modelos de n-gramas

Guía Intuitiva, Explicación Conceptual y Plan de Trabajo en Equipo

Curso de Teoría de la Probabilidad · Quinto Semestre · Licenciatura en Computación

Noviembre de 2026

Propósito de esta guía

Este documento sintetiza el proyecto final de la materia de forma clara, amena e intuitiva. Está pensado para que cualquier integrante del equipo, independientemente de sus conocimientos previos sobre modelos de lenguaje, comprenda de inmediato **qué se busca lograr, por qué tiene sentido probabilístico, cómo se resuelven los retos del proyecto y cómo organizarse en un equipo de 4 personas** para completarlo paso a paso con éxito y sin frustración.

1. ¿De qué trata este proyecto en palabras sencillas?

Imagina que estás escribiendo un mensaje en tu teléfono móvil y escribes la letra:

« q »

Tu intuición y el teclado predictivo de tu teléfono saben de inmediato que la siguiente letra, con casi un 100 % de probabilidad en español, será la letra **« u »**.

Ahora imagina que acabas de escribir:

« la ca »

¿Qué letra viene a continuación? Podría ser una **«s»** (*la casa*), una **«m»** (*la cama*), una **«r»** (*la cara*) o una **«l»** (*la cal*). Hay cierta incertidumbre, pero sabemos con certeza que difícilmente vendrá una **«x»** o una **«w»**.

Este proyecto consiste exactamente en construir una **máquina predictiva que intenta adivinar el siguiente carácter (letra, espacio o signo) de un texto en español**, basándose exclusivamente en el cálculo de probabilidades obtenido a partir de conteos en un libro real.

A la cantidad de caracteres previos que miramos como contexto se le llama en probabilidad el **orden del modelo de n-gramas**:

- **Unigramas ($n = 1$):** El modelo no tiene memoria del pasado; solo predice según la frecuencia general de cada letra en el texto (por ejemplo, sabiendo que la «e» y la «a» son muy comunes en español).
- **Bigramas ($n = 2$):** El modelo mira **la última letra** para predecir la siguiente (ej. qué letra suele seguir a la «q»). Formalmente, es una *Cadena de Markov de primer orden.*
- **Trigramas ($n = 3$):** El modelo mira las **últimas 2 letras** para predecir la tercera.
- **Modelos de orden $n = 4$ y $n = 5$:** El modelo mira las últimas 3 o 4 letras (ya empieza a reconocer raíces y fragmentos enteros de palabras).

2. El objetivo central y la gran pregunta a responder

El proyecto no busca simplemente escribir código, sino responder experimentalmente con datos a la siguiente pregunta de investigación:

«¿Cuánto mejora la predicción del siguiente carácter al aumentar la memoria del modelo (el orden n) y a partir de qué orden deja de compensar por falta de datos?»

La trampa que van a experimentar: A primera vista, uno podría pensar: *«¡Pues démosle al modelo una memoria de 10 letras para que sea súper inteligente!»*. Pero en la práctica ocurre algo crítico:

- Mientras más largo es el contexto que exigimos recordar, **menos veces habrá aparecido esa combinación exacta en el libro**.
- Cuando el modelo se encuentra con una secuencia que nunca vio en su fase de estudio, la probabilidad calculada por frecuencia empírica da exactamente **cero (0)**.
- Como las métricas de evaluación utilizan logaritmos ($\log_2(0) \to -\infty$), asignar probabilidad cero a un evento real **arruina por completo el rendimiento del modelo** (da un error infinito).

Resolver este fenómeno mediante un concepto probabilístico llamado **suavizado** es uno de los aportes centrales del proyecto.

3. Materia prima y herramientas de trabajo

1. **El Corpus (Texto de estudio):** Se utilizará una obra literaria en español de dominio público descargada de Project Gutenberg con al menos **300 000 caracteres** (por ejemplo, una novela clásica). Cada equipo debe elegir una obra distinta y registrar en la primera semana su título, autor, URL y fecha de descarga.
2. **Limpieza y Normalización Obligatoria:**
   - Convertir todo el texto a minúsculas.
   - Colapsar espacios múltiples y saltos de línea repetidos en un solo espacio en blanco.
   - Reemplazar cualquier símbolo fuera del alfabeto elegido por un carácter especial único (por ejemplo `#`). El vocabulario final $\mathcal{V}$ rondará típicamente entre 30 y 45 caracteres.
3. **Partición de los Datos (80 % / 10 % / 10 %):** El libro se divide en tres bloques contiguos en este orden:
   - **80 % Entrenamiento:** De donde el modelo aprende y cuenta las frecuencias de las letras.
   - **10 % Validación:** El conjunto de pruebas intermedias para calibrar los parámetros del modelo (encontrar el mejor valor de suavizado α).
   - **10 % Prueba:** El examen final definitivo. **Se evalúa una sola vez al final del proyecto**.

   Importante: No se mezclan letras ni oraciones al azar: un texto tiene fuerte dependencia entre caracteres vecinos, y barajar filtraría información del entrenamiento a la evaluación.
4. **Herramientas permitidas:** Python 3 estándar con `NumPy`, `pandas` y `Matplotlib` (o R base con `ggplot2`). **No** se permiten librerías que ya implementen modelos de lenguaje (como NLTK o KenLM) ni redes neuronales. Todo el cálculo probabilístico se programa por el equipo.

4. La prueba de cordura: Su red de seguridad inicial

Para garantizar que nadie pierda tiempo buscando errores matemáticos en un texto gigante de 300 000 letras, el profesor diseñó una **prueba de cordura obligatoria** con una palabra de juguete:

`«abracadabra»`

Antes de procesar la novela real, el script `pruebas.py` debe correr sobre `abracadabra` y reproducir con total exactitud las cifras teóricas de la Tabla 1. Si estas cifras coinciden, ¡el equipo puede avanzar con total tranquilidad sabiendo que sus fórmulas y matrices están blindadas!

Magnitud calculada	**Valor esperado exacto**
Conteos de unigramas	a: 5, b: 2, r: 2, c: 1, d: 1
Entropía de las frecuencias de unigramas	2.0404 bits
$\hat{q}(\text{b} \mid \text{a})$ por Máxima Verosimilitud (MLE, bigramas)	0.5
$\hat{q}_\alpha(\text{b} \mid \text{a})$ con suavizado $\alpha = 1$ y $\lvert\mathcal{V}\rvert = 5$	$1/3 \approx 0.3333$
Bits Por Carácter (BPC) del bigrama MLE sobre sus 10 transiciones	0.6 bits
Perplejidad (PPL) correspondiente	1.5157

Tabla 1: Valores esperados obligatorios para la prueba en `pruebas.py`.

5. El camino paso a paso: Las 6 Fases explicadas fácil

Fase 1: Explorar el corpus (¿Qué tan ordenado es el español?)

- **Qué se hace:** Se cuenta cuántas veces aparece cada carácter en el bloque de entrenamiento y se grafica de mayor a menor frecuencia (Gráfica F1 y Tabla T1).
- **Intuición teórica:** Se calcula la **Entropía** $H(\hat{p})$, que mide el nivel promedio de sorpresa o desorden del texto. Se compara con la entropía máxima posible ($\log_2 |\mathcal{V}|$), que representaría un texto generado al azar equiprobable. La diferencia en bits muestra numéricamente cuánta estructura y regularidad posee el idioma español.

Fase 2: Estimar por Máxima Verosimilitud (MLE)

- **Qué se hace:** Se calculan las probabilidades condicionales empíricas para $n = 1, 2, 3, 4, 5$, dividiendo conteos observados:
$$\hat{q}(y \mid h) = \frac{c(h, y)}{c(h)}$$
- **El hallazgo clave:** Al evaluar en validación, se descubre que para $n \geq 2$ aparecen secuencias no observadas en el entrenamiento, asignando probabilidad 0. Esto dispara la proporción de ceros (Tabla T2) y demuestra por qué la MLE pura falla al generalizar.

Fase 3: Suavizado aditivo y calibración del parámetro α

- **Qué se hace:** Para que ninguna combinación tenga probabilidad cero, se suma un valor $\alpha > 0$ a cada celda de conteo (fórmula de Laplace generalizada):
$$\hat{q}_\alpha(y \mid h) = \frac{c(h, y) + \alpha}{c(h) + \alpha|\mathcal{V}|}$$
- **Experimento:** Se prueba una rejilla de valores de $\alpha \in \{0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1\}$ sobre el bloque de validación.
- **Resultado:** Se grafica el error BPC contra α para cada n (Gráfica F2) y se elige el α óptimo que minimice el error de validación.

Fase 4: Evaluación en prueba y comprobación teórica

- **Qué se hace:** Con los mejores α ya fijados en validación, se evalúa **una sola vez** el bloque de prueba.
- **Métricas principales:** Se reporta el **BPC** (Bits por Carácter, menor es mejor) y la **Perplejidad** ($\text{PPL} = 2^{\text{BPC}}$).
- **Comprobación teórica:** Se verifica la *Desigualdad de Gibbs* ($H(p) \leq H(p, q)$) y se grafica el promedio acumulado de sorpresa a lo largo de la prueba, comprobando visualmente cómo se estabiliza tal como predice la *Ley de los Grandes Números*.

Fase 5: Simulación (¡Poner a escribir al modelo!)

- **Qué se hace:** Se genera texto sintético de 300 caracteres con los modelos $n = 2, 3, 5$, partiendo de un mismo contexto inicial.
- **Intuición teórica:** En bigramas ($n = 2$), la matriz de transición permite calcular la *distribución estacionaria* π, demostrando que a largo plazo las frecuencias de letras generadas convergen a las frecuencias reales del texto. Además, se analiza qué propiedades del español capturan las frases generadas y cuáles no.

Fase 6: Cuantificar la incertidumbre (Bootstrap)

- **Qué se hace:** ¿Es el mejor modelo significativamente superior al modelo anterior, o fue solo suerte con este texto? Se divide la prueba en bloques contiguos de 2000 caracteres y se aplica *Bootstrap* (10 000 remuestreos) para construir un intervalo de confianza del 95 % sobre la diferencia de rendimiento.

6. Plan de organización para un equipo de 4 personas

Para trabajar de manera equilibrada, sin cuellos de botella y garantizando que todos dominen el contenido, se propone la siguiente distribución funcional:

Integrante	Rol Principal	Entregables y Tareas Clave
Persona 1	**Ingeniero de Datos y Preprocesamiento**	▪ Seleccionar la obra en Gutenberg y documentar la fuente. ▪ Programar `preparar.py`: lectura, normalización y partición 80/10/10 contigua. ▪ Implementar la Fase 1 (vocabulario, frecuencias y Tabla T1).
Persona 2	**Arquitecto de Modelos Probabilísticos**	▪ Programar `modelos.py`: conteos de n-gramas, matrices condicionales MLE y suavizado aditivo con α. ▪ Programar `pruebas.py` para asegurar que la prueba de `abracadabra` pase sin errores.
Persona 3	**Experimentación y Simulación**	▪ Programar `experimentos.py`: cálculo de BPC, perplejidad y selección del mejor α (Fases 3 y 4). ▪ Programar la generación de texto (Fase 5) y el remuestreo Bootstrap del 95 % (Fase 6). ▪ Exportar tablas (`.csv`) y figuras (`.pdf`/`.png`).
Persona 4	**Líder de Reporte y Reproducibilidad**	▪ Estructurar la plantilla del reporte en LaTeX (10 a 14 págs.) según la Tabla 3 del enunciado. ▪ Redactar el marco teórico formal (apoyándose en el libro de Luis Rincón y notas de S01). ▪ Redactar el `README.md` y asegurar que todo corra en menos de 10 minutos.

Tabla 2: Distribución modular sugerida para un equipo de 4 personas.

Recomendación clave para la defensa oral

El profesor se reserva el derecho de solicitar una **defensa oral aleatoria de 10 minutos** a cualquier integrante. Por ello, se recomienda realizar una breve sincronización semanal de 20 minutos donde cada miembro explique su código y sus gráficos al resto del equipo.

7. Cronograma de trabajo sugerido (4 semanas)

- **Semana 1 (Cimientos):** Elegir la obra de Gutenberg y pedir aprobación; programar la limpieza y partición; implementar y verificar la prueba de `abracadabra`.
- **Semana 2 (Modelado):** Implementar conteos de n-gramas (Fase 2); aplicar suavizado aditivo y encontrar el mejor α con validación (Fase 3). Revisión 1 con el profesor.
- **Semana 3 (Evaluación y Simulación):** Evaluar una sola vez en el bloque de prueba (Fase 4); simular texto (Fase 5); calcular el Bootstrap (Fase 6). Revisión 2 con el profesor.
- **Semana 4 (Redacción y Empaquetado):** Integrar todas las tablas y figuras en el reporte de LaTeX; responder las preguntas de discusión; verificar la reproducibilidad completa en menos de 10 minutos y generar el archivo `.zip`.

8. Lista de comprobación antes de entregar

- ☐ El resumen incluye el mejor n y su BPC de prueba con su intervalo Bootstrap.
- ☐ Cada símbolo y término está definido antes de su primer uso formal.
- ☐ El bloque de prueba se utilizó exactamente una sola vez y el reporte lo declara explícitamente.
- ☐ El script `pruebas.py` pasa sin errores y reproduce exactamente las cifras de `abracadabra`.
- ☐ Las tablas (T1–T4) y figuras (F1–F5) están numeradas, citadas y comentadas con unidades (bits, caracteres).
- ☐ Cada pregunta en cursiva de la sección 4 tiene respuesta en la sección 7 de discusión.
- ☐ Las referencias incluyen la obra literaria usada y el libro del curso.
- ☐ La carpeta comprimida se llama exactamente `P1_EquipoNN_Apellido1_Apellido2.zip`.